



**University of
Nottingham**

UK | CHINA | MALAYSIA

HyVis : TSP Visualisation Tool

Submitted Sept 2024, in partial fulfillment of
the conditions for the award of the degree **MSc Data Science**.

Girija Suresh Dahibhate
20597330

Supervised by Ender Ozcan

School of Computer Science
University of Nottingham

I hereby declare that this dissertation is all my own work, except as indicated in the
text:

Signature: Girija Suresh Dahibhate

Date: 06/09/2024

I hereby declare that I have all necessary rights and consents to publicly distribute this
dissertation via the University of Nottingham's e-dissertation archive.

Abstract

This dissertation explores the development of an interactive visualisation tool for solving the Traveling Salesman Problem (TSP) by integrating Reinforcement Learning (RL) and Simulated Annealing (SA). The tool leverages dynamic cooling schedules and heuristic selection to optimise TSP solutions while providing real-time, user-friendly visualisations of the optimisation process. By addressing gaps in the literature, particularly the need for tools that combine high-quality optimisation with intuitive user interfaces, this project presents a novel approach to TSP optimisation.

The tool’s performance was evaluated through multiple experiments, assessing the effectiveness of different cooling schedules—geometric, linear, and Lundy-Mees—in terms of solution quality and convergence speed. The visualisations, including route maps, heuristic usage metrics, and reward progress plots, enhance user interaction and facilitate a deeper understanding of the optimisation process.

While the tool performs effectively for small to medium-sized TSP instances, challenges remain in scaling to larger problems. Future work will focus on expanding the heuristic library and improving the tool’s scalability to handle more complex scenarios.

Keywords: *Traveling Salesman Problem (TSP), Reinforcement Learning (RL), Simulated Annealing (SA), Optimization Algorithms, Cooling Schedules, Interactive Visualization, Heuristic Selection, Route Optimization, Exploration and Exploitation, NP-hard Problems, User-Centric Design, Dynamic Feedback, Metaheuristics, Convergence, Real-Time Interaction, Plotly, Dash*

Acknowledgements

I would like to express my heartfelt gratitude to my family and friends for their unwavering support, encouragement, and guidance throughout this journey. Their belief in me has been invaluable. I would also like to extend my deepest appreciation to my dissertation supervisor for their consistent guidance, insightful feedback, and support, which were crucial to the completion of this project. Without the collective efforts and encouragement of those around me, this work would not have been possible.

Contents

Abstract	i
Acknowledgements	ii
1 Introduction	1
1.1 Motivation	2
1.2 Aims and Objectives	3
2 Literature Review	5
2.1 Simulated Annealing: Cooling Schedules, Adaptations, and Applications .	6
2.2 Reinforcement Learning and Heuristic Integration	8
2.3 Modular and Flexible Heuristic Framework	9
2.4 Visualisation of Optimisation Processes and Uncertainty	9
3 Methodology	12
3.1 Reinforcement Learning (RL) Agent Implementation	13
3.2 Simulated Annealing (SA) for Cooling Schedules	14
3.3 Tool Workflow and Interaction	15
3.3.1 Visualisation Techniques	16
3.4 Quantitative Metrics and Evaluation	17
3.4.1 Quantitative Metrics:	17
3.4.2 Preliminary Results and Benchmarks:	18
3.4.3 Qualitative Insights	19
4 Implementation	21
4.0.1 System Architecture	21
4.0.2 TSP Problem Domain Setup	22
4.0.3 Reinforcement Learning Agent	23
4.0.4 Simulated Annealing and Cooling Schedules	25
4.0.5 Cooling Schedules Implementation:	26

4.0.6	Tool Workflow and Interaction	26
4.0.7	Visualisation Implementation	27
4.0.8	Testing and Performance Evaluation	28
4.0.9	Challenges and Limitations	28
5	Discussion and Evaluation	30
5.0.1	Interpretation of Key Findings	30
5.0.2	Presentation of Test Cases	31
5.0.3	Evaluation of Quantitative Results	35
5.0.4	Evaluation of Qualitative Results	37
5.0.5	Improvements Based on Feedback	37
5.0.6	Challenges and Limitations	38
5.0.7	Directions for Future Research	39
6	Conclusion	42
	Bibliography	43

List of Figures

5.1	TSP Optimised Route for Test Case 5.	33
5.2	Heuristics Utilization for Test Case 5.	33
5.3	Effective Utilisation Rate of Heuristics for Test Case 5.	34
5.4	Reward Progress Plot for Test Case 5	34
5.5	Reward Progress Plot for Test Case 5	34
5.6	Interactive "i" button displaying information and tooltips for heuristic and parameter descriptions in the HyVIS: TSP visualisation Tool.	35
5.7	Summary Table for Test Cases Performed	36
5.8	Performance of each cooling schedule for Test 5 and Test 6	37

Chapter 1

Introduction

The Traveling Salesman Problem (TSP) is a classical optimisation problem that has captivated researchers for decades due to its theoretical complexity and practical significance. It is a problem deeply rooted in fields such as logistics, transportation, and network design, where finding the most efficient route is paramount [11]. For example, optimising delivery routes for logistics companies or scheduling tasks in manufacturing can be modeled as TSP, making it a cornerstone problem in operations research. The challenge posed by TSP lies in its NP-hard nature [11], meaning that as the number of cities increases, the difficulty of finding an optimal solution grows exponentially. This has made TSP not only a benchmark for testing optimisation algorithms but also a critical problem in real-world applications where large instances must be solved efficiently.

Despite extensive research on TSP, traditional methods such as dynamic programming and branch-and-bound have proven to be computationally prohibitive for large-scale problems [5]. These methods, while effective for smaller instances, struggle with scalability, leading to impractically long computation times as the problem size increases. Moreover, most existing TSP solvers are designed with a primary focus on optimisation performance, often neglecting the importance of how solutions are presented to the user. In real-world applications, decision-makers require more than just optimal solutions; they need tools that provide transparency, allowing them to understand the optimisation process, make informed decisions, and explore different scenarios.

This dissertation is driven by the recognition of two critical gaps in the current state of TSP research: the computational difficulty of solving large instances and the lack of user-friendly, interactive tools that enhance understanding of the optimisation process. While Reinforcement Learning (RL) and Simulated Annealing (SA) heuristics have been explored in the context of TSP, particularly within frameworks like HyFlex [8], where selection hyper-heuristics are tested across multiple domains, there remains a significant

opportunity to develop tools that integrate these advanced heuristics with robust visualisation capabilities. By doing so, this research tries to respond to the challenge of solving TSP efficiently and gives more emphasis to the user in understanding and being able to interact with the solutions.

The problem-solving tool (HyVis :TSP Visualisation Tool) presented in this study has aimed to serve not only the purpose of more effective problem-solving for TSP by leveraging the strengths of RL and SA combined but also to show dynamic visual feedback so users can appreciate the impact of different heuristics and parameter adjustments in real time. This novel combination of advanced heuristics and visualization has been prepared to fill in the gap between raw computational power and a user-centric design for much better and more holistic resolution of such complex optimisation problems.

The project seeks to demonstrate how advanced heuristic methods can be integrated and made more accessible through thoughtful visualisation design. In broad terms, this research tries to bring state-of-the-art developments in the area of heuristic optimisation and to provide practical solutions for real-world problems that are complex in nature. This introduction is intended to take the reader one step at a time from a general topic, TSP, to the specific focus of this research and indicate the importance of the issue, point out the gaps in current research, and outline the purpose and scope of the study.

The aim of this project, the objectives, and the motivation will be fully elaborated in the subsequent sections, including the methodologies employed to develop these areas, explaining how each facet of the research contributes to the solution as a whole.

1.1 Motivation

Traditional approaches to solving TSP, such as dynamic programming and branch-and-bound, often struggle with scalability, leading to long computation times and limited practical application for large-scale problems [5].

Moreover, most existing TSP solvers prioritize optimisation performance without adequately considering how the solutions are presented to users. In real-world applications, decision-makers often require not just the best possible solutions but also a clear un-

derstanding of the underlying optimisation process to make informed decisions, adjust parameters, and evaluate different scenarios.

This project is motivated by the need to address two critical challenges: the computational difficulty of solving large TSP instances and the lack of user-friendly, interactive tools that provide transparency in the optimisation process. While RL and SA have been applied to TSP in various contexts, the novelty of this project lies in its focus on visualisation. By integrating these advanced heuristics into an interactive visualisation tool, this project aims to create a solution that not only efficiently addresses TSP but also significantly enhances user engagement and understanding. The tool has been created to deliver dynamic visual updates, enabling users to explore the impact of various heuristics and parameters on the optimisation process in real-time.

1.2 Aims and Objectives

The aim of this project is to develop a comprehensive and interactive visualisation tool that efficiently solves the Traveling Salesman Problem (TSP) by integrating Reinforcement Learning (RL) and Simulated Annealing (SA) heuristics. The tool is designed to enhance the user's understanding of the optimisation process by providing dynamic visual updates on heuristic effectiveness, solution quality, and the impact of algorithmic parameter adjustments.

Objectives :

The objectives of this project include developing an integrated heuristic framework that features a utility-based RL agent capable of dynamically selecting and adjusting SA-inspired heuristics to efficiently solve TSP instances. The framework aims to balance exploration and exploitation while emphasizing the visualisation of heuristics to enhance user understanding. Another key objective is to incorporate and evaluate different cooling schedules, such as geometric, linear, and Lundy-Mees, within the framework. These schedules will be assessed based on their impact on solution quality and convergence rates, with a focus on effectively visualising them for user analysis.

Additionally, the project seeks to design and implement interactive visualisation components using Plotly Dash, providing real-time updates on the optimisation process. Visualisations will include TSP routes, heuristic utilization rates, reward progress, and the effectiveness of different cooling schedules, all aimed at improving user comprehension of these processes.

The tool's performance will be evaluated through experiments, measuring the effectiveness of the integrated RL and SA approach in solving TSP instances. Metrics such as solution quality, convergence speed, and the added value of visualisation features will be key to understanding the tool's impact on user experience.

Furthermore, the project addresses the identified research gaps by integrating RL, SA, and advanced visualisation techniques. This integration tackles the need for tools that combine high-quality optimisation with user-friendly, interactive interfaces, while acknowledging prior work in RL-SA integration.

Chapter 2

Literature Review

The Traveling Salesman Problem (TSP) is a classical optimisation problem that has been extensively studied due to its computational complexity and practical applications in various fields such as logistics, manufacturing, and routing. The problem involves finding the shortest possible route that visits a set of cities exactly once and returns to the origin city. Despite its simplicity in formulation, TSP is NP-hard, making it a challenging problem to solve optimally as the number of cities increases. To address this challenge, this dissertation focuses on developing a visualisation tool that integrates Reinforcement Learning (RL) and Simulated Annealing (SA) heuristics. This tool aims to not only solve TSP efficiently but also to provide an intuitive and interactive visualisation of the solutions, enhancing user understanding and exploration of the problem space. While traditional methods like dynamic programming and branch-and-bound have been applied to TSP, they are often computationally expensive and impractical for large instances. Branch-and-bound systematically explores all possible solutions but uses bounding to prune large portions of the search space that cannot contain the optimal solution. Although this helps reduce the search effort, it still faces scalability challenges for larger instances of TSP, as shown in hybrid approaches explored by [5], where hybrid Branch-and-Bound (BB) methods were outperformed by evolutionary algorithms and other modern heuristics. Recent research has explored the use of heuristic approaches, such as SA, and machine learning techniques, like RL, to find approximate solutions more efficiently. However, there is a noticeable gap in tools that combine these methods with robust visualisation capabilities, allowing users to both observe and interact with the solution process in real-time. The existing literature highlights several key areas that this dissertation seeks to address:

1. **Lack of Integrated Approaches:** There is limited research on tools that effectively integrate RL and SA for TSP, particularly with a focus on visualisation.

2. **Visualisation as an Afterthought:** Most existing solutions prioritize optimisation performance without considering how the results are presented to the user, leading to a gap in user-centric design.

The dissertation directly tackles these identified gaps by developing a visualisation tool that not only integrates RL and SA heuristics for solving TSP but also places a strong emphasis on user experience. The visualisation tool is designed to provide an intuitive and interactive visualisation of the solutions, enabling users to explore the problem space and observe the optimisation process in real-time. By doing so, the research aims to create a comprehensive solution that enhances both the efficiency of TSP solving and the accessibility of the results through effective visualisation.

2.1 Simulated Annealing: Cooling Schedules, Adaptations, and Applications

Simulated Annealing (SA) is a widely used metaheuristic inspired by the annealing process in metallurgy. It involves probabilistic acceptance of worse solutions to escape local optima, with the probability controlled by a temperature parameter that gradually decreases according to a cooling schedule. The selection of an appropriate cooling schedule is critical to the performance of SA, especially in complex optimisation problems like TSP. [3] conduct a landscape-based analysis comparing the performance of Fixed Temperature Algorithms (FTA) and traditional Simulated Annealing (SA) across different optimisation landscapes. The study highlights conditions under which FTA can outperform traditional SA, particularly in random instances versus structured instances. While the paper primarily focuses on problems like the Quadratic Assignment Problem (QAP) and Permutation Flowshop Problem (PFSP), its insights into landscape features and algorithmic performance are highly relevant to TSP. However, the paper does not explore these findings specifically in the context of TSP, presenting a research gap that can be addressed by applying the insights gained to the TSP domain.[7] investigate the effectiveness of different SA cooling schedules (geometric, linear, logarithmic, and adaptive

AARTs) in solving the RNA design problem. Their findings indicate that logarithmic cooling schedules are particularly effective in difficult problem instances, while geometric schedules show robustness across various scenarios. Although the study is focused on RNA design, the detailed exploration of cooling schedules offers valuable insights that can be applied to optimising SA for TSP. As such a research gap exists in exploring how these cooling schedules perform in TSP and whether adaptive methods like AARTs could improve SA's adaptability in dynamic optimisation scenarios. The Enhanced List-Based Simulated Annealing (ELBSA) algorithm, introduced by [12], addresses the challenges of applying SA to large-scale TSP instances. By utilizing list-based data structures, ELBSA efficiently manages and evaluates candidate solutions, reducing computational overhead. This method is particularly relevant for large and complex problem instances where traditional SA might struggle with scalability. The ELBSA approach demonstrates significant improvements in both solution quality and computational efficiency, making it a valuable adaptation of SA for TSP. However, further research could explore the integration of RL to dynamically adjust parameters like temperature and move selection based on real-time feedback. [6] explore the application of Simulated Annealing to the Multiple Traveling Salesman Problem (MTSP), a variant of TSP involving multiple salesmen. The study focuses on the implementation and tuning of SA parameters, such as initial temperature and cooling coefficient, and how these affect solution quality and computational efficiency. The insights gained from this paper are particularly relevant for optimising SA in TSP, where similar trade-offs between exploration and exploitation must be managed. However, there is a gap in exploring how these findings translate to the standard TSP and whether integrating RL could enhance SA's performance in dynamic optimisation contexts.

Although existing studies offer valuable insights into the application of Simulated Annealing (SA) in various optimisation problems, several research gaps and limitations remain. Most research has focused on variants of the Travelling Salesman Problem (TSP) or other optimisation problems like RNA design and the Quadratic Assignment Problem (QAP), leaving a need to apply these findings specifically to TSP to assess the effectiveness of the same cooling schedules and parameter settings. Additionally, limited research has ex-

plored the potential integration of Reinforcement Learning (RL) with SA, where RL could dynamically adjust SA parameters, such as temperature schedules, or guide the search process based on real-time feedback, potentially improving performance. Additionally, these studies lack consideration for visualising the optimisation process or enabling user interaction with the results. Developing visualisation strategies that allow users to explore the effects of different cooling schedules on solution quality could enhance the utility of SA in the TSP domain.

2.2 Reinforcement Learning and Heuristic Integration

Reinforcement Learning (RL) is a machine learning approach where an agent learns to make decisions by interacting with an environment, receiving rewards or penalties based on its actions. When applied to optimisation problems like TSP, RL can be used to dynamically select or adjust heuristics during the search process, enhancing the algorithm's ability to adapt to different problem instances. [13] introduce a reinforcement learning-based approach within a hyper-heuristic framework aimed at solving complex scheduling problems, such as examination timetabling. RL is used to dynamically select low-level heuristics based on their past performance, while the Great Deluge algorithm serves as a move acceptance strategy to avoid local optima. This approach is particularly relevant to TSP, where the complexity of the solution space necessitates dynamic adaptation and strategic exploration. However, there is a gap in directly applying these findings to TSP and exploring how RL could be integrated with SA to further enhance the optimisation process.

There is a significant research opportunity in exploring the integration of Reinforcement Learning (RL) with Simulated Annealing (SA), particularly in using RL to adapt cooling schedules or guide the search process based on feedback from the solution space.

2.3 Modular and Flexible Heuristic Framework

Heuristic frameworks provide a structured approach to developing and testing optimisation algorithms across various problem domains. A modular framework allows for the separation of problem-specific components from general-purpose algorithms, enabling easy adaptation and testing across different optimisation problems. [8] present the HyFlex framework, a modular platform designed to facilitate the development and benchmarking of heuristic algorithms across various optimisation problems, including TSP. HyFlex’s modularity, which separates problem-specific components from general-purpose algorithms, allows for easy adaptation and testing across different domains. For TSP research, the HyFlex framework’s flexibility is invaluable, enabling systematic comparison of different heuristic strategies. However, further research could explore how this modular approach can be adapted specifically for TSP, particularly in integrating RL and SA.

Despite HyFlex offering a strong framework for cross-domain heuristic search, there is a notable gap in its application to the Travelling Salesman Problem (TSP), particularly in the integration of advanced techniques such as Reinforcement Learning (RL) and Simulated Annealing (SA). Additionally, the scalability of heuristic frameworks like HyFlex to handle large-scale TSP instances remains an area that requires further investigation.

2.4 Visualisation of Optimisation Processes and Uncertainty

Advanced data visualisation techniques are essential for interpreting and analysing the results of optimisation processes. Visualisation tools can help users explore the solution space, understand the impact of different parameters, and gain deeper insights into the optimisation process. [2] describe the development of the reV Dashboard, an interactive tool designed to visualise uncertainties in renewable energy potential. Built using the Plotly Dash framework, the dashboard provides various interactive features, such as geographic maps and parallel coordinate plots, allowing users to explore and interpret

complex datasets. Although focused on renewable energy, the principles of uncertainty visualisation and interactive data exploration presented in this paper are directly applicable to the TSP visualisation tool. However, there is a gap in adapting these techniques specifically to TSP, particularly in visualising the impact of different cooling schedules and optimisation strategies on solution quality. [1] introduce an interactive visualisation tool designed to detect and analyse outliers in eye-tracking data, particularly focusing on scanpaths. The tool leverages visualisation techniques such as adjacency matrices, visual attention maps, and gaze plots to compare scanpaths and identify anomalies. While the primary application is in eye-tracking data, the visualisation techniques discussed in this paper can be adapted to TSP, especially in identifying and analysing outliers in the solution space. However, there is a gap in directly applying these visualisation techniques to TSP and exploring how they can be integrated into a comprehensive visualisation tool for optimisation. [10] emphasise the critical role of mathematical optimisation in enhancing data visualisation and visual analytics. This paper discusses how optimisation techniques can be applied throughout the visualisation pipeline—from data processing and transformation to interactive exploration and analysis—to improve the effectiveness of visualisation tools. In the context of TSP, these principles are essential for creating a tool that not only solves the optimisation problem but also presents the results in a clear, actionable manner. Integrating advanced visualisation techniques into the TSP tool may potentially enable users to explore the solution space more effectively, understand the impact of different parameters, and gain deeper insights into the optimisation process. [4] explore the development of a visualisation system designed to optimise stock portfolios using Quantum-inspired Tabu Search (QTS) algorithms. While the focus is on financial portfolio management, the principles of using advanced metaheuristic algorithms and integrating them into a user-friendly visualisation tool are directly relevant to TSP visualisation. The paper’s emphasis on real-time interaction, push notifications, and scalability can inspire similar features in a TSP tool, ensuring it is both user-centric and efficient. However, the domain-specific focus presents a research gap in adapting these principles to TSP, particularly in integrating SA and RL.

There is a research gap in the adaptation of advanced visualisation techniques to the domain of the Travelling Salesman Problem (TSP), especially in terms of illustrating the effects of various optimisation strategies and identifying outliers within the solution space. Additionally, further research is required to investigate how users interact with visualisation tools, particularly in gaining a deeper understanding of the underlying dynamics of the optimisation process.

In summary, a review of the existing literature has explored the integration of Reinforcement Learning (RL) and Simulated Annealing (SA) for solving the Traveling Salesman Problem (TSP) and emphasised the critical role of visualisation in enhancing user interaction and understanding. The review identified significant gaps in the existing literature, particularly the limited research on tools that combine RL and SA with robust visualisation capabilities.

Through the analysis of ten key papers, it was found that while RL and SA offer promising approaches to tackling the computational challenges of TSP, there is a lack of integrated solutions that prioritize both optimisation performance and user-centric design. This gap emphasizes the necessity for a comprehensive tool that not only solves TSP efficiently but also presents the solutions in an intuitive and interactive manner.

The insights gained from these studies will inform the development of a novel visualisation tool that effectively integrates RL and SA, addressing the identified gaps and advancing the state-of-the-art in TSP optimisation. By doing so, this dissertation aims to contribute significantly to both the theoretical and practical aspects of TSP, offering a solution that enhances both the efficiency and accessibility of TSP-solving methods.

Chapter 3

Methodology

This section outlines the methodology used to achieve the research objectives of this study. Given the importance of TSP in various industries, there is a significant demand for tools that can solve large instances efficiently and provide users with a clear understanding of the optimisation process. This research addresses the need for such a tool by integrating Reinforcement Learning (RL) and Simulated Annealing (SA) heuristics into an interactive visualisation framework. The underlying assumption of this study is that combining RL and SA within a user-friendly visualisation tool will enhance both the efficiency of solving TSP and the user's ability to interact with and understand the solution process.

This study builds on existing research in the field of optimisation and visualisation, drawing inspiration from the work of [3], who demonstrated the effectiveness of integrating different heuristic methods in solving complex optimisation problems. By extending these principles to TSP and focusing on user interaction, this research aims to develop a tool that not only produces high-quality solutions but also facilitates user engagement and exploration of the solution space. This research adopts a mixed-methods approach, combining quantitative analysis of the tool's optimisation performance with qualitative assessment based on iterative development and user testing. The choice of a mixed-methods approach is justified by the need to evaluate both the computational efficiency of the tool and its usability in real-world scenarios.

The quantitative component focuses on measuring the impact of different SA cooling schedules on the quality of the TSP solutions and the convergence speed of the optimisation process. This aspect of the research is inspired by studies done by [7] work on cooling schedules, which highlights the importance of temperature control in metaheuristic algorithms like SA. Metrics such as total distance, convergence rate, and computational efficiency are quantitatively assessed across various problem instances and cooling sched-

ules.

The qualitative component, on the other hand, examines how the visualisation tool supports user understanding and interaction during the optimisation process. This aspect of the study is informed by the work of [2], who emphasized the importance of effective visualisation in complex data analysis. By evaluating the user interface and interaction design, this research seeks to ensure that the tool is not only computationally powerful but also accessible and intuitive for users.

3.1 Reinforcement Learning (RL) Agent Implementation

The RL agent, named SimpleUtilityBasedAgent, is central to the tool’s optimisation process. It selects actions, or heuristics, based on their utility, which is dynamically updated according to the success of previous actions. Rooted in reinforcement learning principles, the agent learns to make decisions by maximizing rewards over time, with the goal of identifying the most effective heuristics for reducing the total distance of the TSP route and prioritizing them in future iterations.

Each heuristic is associated with a utility value that is adjusted based on the heuristic’s performance. The agent employs three adaptation strategies—additive, divisional, and root—to fine-tune these values. The additive strategy rewards heuristics that perform well by increasing utility by a fixed amount, while the divisional strategy rewards them proportionally by multiplying the utility by a factor. The root strategy moderates these changes, applying a square root transformation to prevent overly favouring a single heuristic too quickly and ensuring a balance between exploration and exploitation. This is particularly useful for avoiding premature convergence, where the agent might become too reliant on one heuristic and fail to explore other potentially effective options. The agent’s learning process involves updating utility values after each action. If an action results in a better TSP solution, typically measured by a reduction in total route cost, the utility of that action increases. Conversely, if it doesn’t lead to improvement, the

utility is decreased accordingly. This continuous learning process enables the agent to optimize its choice of heuristics, making the tool more effective at solving diverse TSP instances. This approach is particularly beneficial in dynamic environments, where the problem characteristics may change over time, requiring the agent to adapt continuously.

3.2 Simulated Annealing (SA) for Cooling Schedules

Simulated Annealing (SA) is a probabilistic technique inspired by the annealing process in metallurgy, where a material is heated and then slowly cooled to remove defects and achieve a more stable structure. In the context of optimisation, SA is used to escape local optima by allowing the algorithm to accept worse solutions with a certain probability, which decreases as the “temperature” decreases. This temperature is controlled by a cooling schedule, which dictates how quickly the probability of accepting worse solutions diminishes over time.

The tool implements several cooling schedules, each with its own mathematical formulation and characteristics. Geometric cooling decreases the temperature multiplicatively by a constant factor at each step, making it simple and effective, especially in smoother problem spaces where the algorithm can gradually narrow its focus on promising regions. Linear cooling reduces the temperature by a fixed amount at each step, providing a slower cooling process. This allows for more extensive exploration of the solution space, making it useful in situations with many local optima, as it offers a gradual reduction in accepting worse solutions. The Lundy-Mees cooling schedule combines aspects of both geometric and linear cooling, offering a more complex decay function that balances exploration and exploitation. This method is particularly well-suited for complex optimisation problems where the solution space is highly irregular and requires more nuanced temperature control. The integration of SA with RL provides a powerful combination for solving TSP. While the RL agent selects heuristics based on learned utility values, the SA-inspired cooling schedule modulates the search process by adjusting the likelihood of accepting suboptimal moves. This balance between exploration and exploitation is crucial for finding high-quality solutions in complex optimisation problems like TSP. The cool-

ing schedule ensures that the algorithm does not get trapped in local optima too early, allowing the RL agent to continue exploring the solution space effectively.

3.3 Tool Workflow and Interaction

The tool’s workflow starts with the initialization of the TSP problem domain, where cities are randomly distributed on a plane. The RL agent selects a heuristic based on its utility values, which is then applied to modify the current solution. The SA cooling schedule determines whether the new solution is accepted. This process repeats iteratively, with the RL agent updating its utility values and the SA cooling schedule gradually reducing the temperature. Each step allows the tool to explore different regions of the solution space while gradually honing in on the most promising areas. The iterative nature ensures that the tool adapts to changes in the problem domain, improving the solution over time. NGrok, a reverse proxy service, was utilized during the development phase to create secure tunnels to the locally hosted tool [9], enabling external access without requiring complex network configurations such as port forwarding. By generating a public URL, NGrok allowed to interact with the tool remotely in a secured way, facilitating real-time testing and feedback. This service proved invaluable in web development and testing scenarios, as it ensured remote accessibility and control without intricate setup. The data flow within the tool is managed by Dash’s callback functions, which handle user inputs, process the optimisation results, and update visualisations. These callbacks manage the state of the application, ensuring that each component receives the correct data at the appropriate time. The backend processes the TSP problem, runs the RL and SA algorithms, and sends the results to the frontend for visualisation. This separation between backend and frontend ensures the tool remains responsive and efficient, even when working with large datasets or complex configurations. The tool is designed for interactivity, allowing users to experiment with different parameters and observe their effects on the TSP solution. Users can adjust variables such as the number of cities, episodes, maximum steps, Learning Rate, Discount Factor , Exploration Rate , Initial Temperature, Temperature Decay Rate. As they modify these parameters, the tool dynamically updates the visualisations, providing

real-time feedback on how changes affect the solution. This interactivity is essential for helping users gain a deeper understanding of the optimisation process and explore various scenarios to find the best solution.

3.3.1 Visualisation Techniques

The visualisation component of the tool is built using Plotly Dash, selected for its ability to create interactive, web-based visualisations. The design follows principles of effective data visualisation, such as clarity, consistency, and accessibility, with the aim of presenting complex optimisation data in a way that is easy to interpret, even for users without a technical background.

Several customizations were implemented to enhance the visual appeal and usability of the tool. A pastel color scheme was chosen to make the charts and graphs easier to read and less visually overwhelming. Different types of visualisations—such as bar charts, pie charts, and line graphs—were carefully chosen to convey specific aspects of the optimisation process. For instance, bar charts illustrate the effective utilisation rates of different heuristics, while line graphs track the convergence of the TSP solution over time.

The user interface (UI) was designed with a focus on user experience, aiming to create an intuitive and easy-to-navigate environment. Users can input parameters, run the optimisation process, and view the results without requiring extensive technical knowledge. The interface includes clear labels, tooltips, and a logical layout that guides users through each step of the process.

Given the complexity of the data being presented, clarity was a key consideration in the UI design. Results, such as the TSP route, heuristic utilisation rates, and convergence plots, are displayed in separate sections with enough spacing to avoid clutter. This separation allows users to focus on each aspect of the optimisation process individually, making the results easier to interpret.

To further enhance user engagement, the UI includes interactive elements like buttons to start the optimisation process and arrows to adjust parameter values. These interactive components allow users to explore how different settings impact the TSP solution in real

time. Tooltips provide additional context for each element, helping users understand the effects of their choices on the outcome.

3.4 Quantitative Metrics and Evaluation

3.4.1 Quantitative Metrics:

The total distance travelled by the Traveling Salesman in the optimised route is the primary metric for evaluating the quality of the solution, as the main objective of the TSP is to minimize this distance. A lower total distance indicates a more efficient route, directly reflecting the performance of the optimisation algorithm. This metric was measured across multiple runs and cooling schedules to assess the tool’s consistency and efficiency. By comparing the total distance across different configurations, insights were gained into which cooling schedules and heuristics provided the best results.

The utilisation rate of heuristics represents the percentage of iterations in which each heuristic was selected during the solution process. This metric offers insight into the behavior of the RL agent, showing which heuristics are favored under specific conditions or cooling schedules. utilisation rates were compared across different cooling schedules to understand each heuristic’s role in the optimisation process. This analysis helped identify which heuristics were most effective and under what circumstances they were chosen by the RL agent.

The effective utilisation rate measures how often a selected heuristic led to an improvement in the solution by reducing the total distance. This metric distinguishes between the simple usage of a heuristic and its actual contribution to the optimisation process. By analyzing the effective utilisation rates, the most impactful heuristics were identified, guiding further tuning of the tool to prioritize those that consistently produced better results.

Reward progress tracks the change in the reward value assigned by the RL agent during the optimisation process. This reward is typically linked to improvements in the objective function, such as a reduction in total distance. Monitoring reward progress provides insight into how the agent learns and adapts during optimisation, as well as reflecting the

algorithm’s convergence behavior. Reward progress was plotted to visually analyze how quickly and effectively the agent optimised the TSP route, providing a clear representation of the learning process over time.

Finally, the convergence rate measures how quickly the optimisation algorithm approaches a stable solution, indicated by changes in the total distance over iterations. A faster convergence rate suggests a more efficient algorithm capable of reaching near-optimal solutions swiftly. Convergence rates were monitored and compared across different cooling schedules to determine the most efficient and reliable convergence approach. This metric was critical in evaluating the tool’s overall efficiency, indicating how quickly the algorithm could find high-quality solutions.

3.4.2 Preliminary Results and Benchmarks:

Initial tests demonstrated that the tool consistently produced optimised routes with minimal variance in total distance across multiple runs, indicating reliable performance. This consistency was essential for validating the tool’s effectiveness and ensuring it could be trusted to deliver high-quality solutions across different scenarios.

The utilisation and effective utilisation rates revealed distinct patterns, showing that certain heuristics were more effective under specific cooling schedules. These patterns provided valuable insights into how the RL agent adapts to various scenarios and how the choice of cooling schedule impacts the overall optimisation process.

The reward progress plots generally showed a positive trend, indicating successful learning and continuous improvement in the TSP route. Additionally, the convergence plots confirmed that the tool efficiently found near-optimal solutions within a reasonable number of iterations. These results demonstrated the effectiveness of the RL-SA integration and established a solid foundation for further refinement of the tool.

In terms of benchmarks, the impact of different cooling schedules—geometric, linear, and Lundy-Mees—was thoroughly benchmarked against one another. Each cooling schedule has distinct characteristics that influence the algorithm’s behaviour. The geometric

schedule, which reduces the temperature by a constant factor, was found to be effective in smooth problem spaces, where it led to quicker convergence. However, the linear schedule, which decreases the temperature by a fixed amount, allowed for more gradual exploration, making it particularly effective in complex problem spaces with numerous local optima. The Lundy-Mees cooling schedule, which combines elements of both geometric and linear approaches, struck a balance between exploration and exploitation, proving highly effective in handling more irregular and complex TSP instances.

These comparisons revealed that the effectiveness of a cooling schedule is highly dependent on the problem size and complexity. For simpler TSP instances, geometric cooling often led to faster solutions, while in more complex cases, linear and Lundy-Mees schedules provided better overall performance. This detailed benchmarking provided valuable insights into the adaptability of the tool and how different cooling schedules can be tuned to enhance performance based on the specific characteristics of the optimisation problem being addressed.

3.4.3 Qualitative Insights

The development of the tool followed an iterative process, where each phase of implementation was accompanied by thorough testing, debugging, and refinement. This approach was crucial for identifying and addressing issues early in development, ensuring the final tool was both robust and user-friendly.

The initial development focused on implementing the core components of the RL and SA algorithms, ensuring they functioned correctly and produced valid, optimised routes for the TSP. Extensive testing was conducted to verify that these algorithms were properly implemented and could reliably solve the problem. As routes and metrics were generated, attention shifted to refining the visualisations. The early visual outputs lacked clarity, prompting the addition of more descriptive labels, legends, and titles in the graphs. The color scheme was adjusted to a pastel palette to enhance visual appeal and readability. These refinements were necessary to present complex data in a manner that was easy to understand and interpret.

The user interface also underwent significant improvement after initial testing revealed that it was not as intuitive as desired. Input fields and controls were reorganized to improve layout and flow, with parameters such as the number of cities, episodes, and cooling schedules restructured for easier access. These changes were essential for making the tool accessible to users of varying technical expertise.

During development, several issues emerged, particularly with callback functions that failed to update graphs as expected. These problems were resolved through careful debugging and restructuring how outputs were generated and returned, ensuring that all lists and variables were correctly referenced. This phase of testing and debugging was crucial for ensuring the tool's functionality and reliability.

As testing progressed, new features were added, including detailed reward progress and convergence plots, which provided deeper insights into the optimisation process. These additions enhanced the tool's ability to explain the behavior of the algorithms over time, making it more valuable for users seeking to understand the intricacies of the optimisation process.

The mixed-methods approach used in the tool's development was justified by the need to evaluate both its computational performance and usability. The integration of RL and SA, supported by [13] and adapted for TSP based on the findings of [7], was critical for addressing the problem. The use of visualisation tools, as recommended by [2] and [10], was emphasized to tackle the gap in user-centric design identified in the literature review.

Chapter 4

Implementation

The primary goal of this project is to develop an advanced visualisation tool for solving the Traveling Salesman Problem (TSP) by integrating Reinforcement Learning (RL) and Simulated Annealing (SA) heuristics. The tool not only aims to optimise the solution to the TSP but also emphasizes user interaction through dynamic and intuitive visualisations. By providing real-time feedback and clear graphical representations, the tool enables users to observe and understand the optimisation process, making it a valuable asset for both researchers and practitioners in the field.

4.0.1 System Architecture

The system architecture is designed around three core components:

Reinforcement Learning (RL) Agent: The RL agent, `SimpleUtilityBasedAgent`, is responsible for selecting and applying heuristics to improve the TSP solution iteratively. It learns which heuristics are most effective through a utility management system and adapts its strategy accordingly.

Simulated Annealing (SA) Schedules: SA schedules control the exploration-exploitation trade-off by adjusting the "temperature" parameter, which influences the agent's likelihood of accepting suboptimal solutions. Three cooling schedules—Geometric, Linear, and Lundy-Mees—are implemented to guide this process.

Visualisation Interface: Built using Dash and Plotly, the interface provides real-time visual feedback on the optimisation process, displaying the TSP route, heuristic utilisation, reward progress, and more. This interface is designed to be interactive, allowing users to adjust parameters and observe the impact on the solution dynamically. Additionally, the interface features interactive tooltips accessed via 'i' buttons adjacent to input fields and visual components. These tooltips offer contextual information and guidance,

enhancing usability and helping users understand the significance of various parameters and visualisations.

4.0.2 TSP Problem Domain Setup

The `TSPProblemDomain` class is central to the TSP simulation. It initialises a set of cities randomly placed in a 2D space and manages potential solutions to the problem. The class includes methods for initialising solutions, applying heuristics, and calculating the total distance of a route. The class has been enhanced to allow the use of predefined cities, ensuring consistency when comparing different cooling schedules.

Listing 4.1: `TSPProblemDomain` class implementation

```
class TSPProblemDomain:
    def __init__(self, cities=None, num_cities=10):
        if cities is not None:
            self.cities = cities
        else:
            self.cities = np.random.rand(num_cities, 2)
        self.num_cities = len(self.cities)
        self.solutions = [None] * 2

    def initialiseSolution(self, index):
        self.solutions[index] = np.random.permutation(self.num_cities)
```

The constructor `__init__` accepts a `cities` parameter. If provided, it uses the predefined set of cities; otherwise, it generates a random set. This enhancement allows for consistent city coordinates across different cooling schedules, facilitating fair comparisons. By passing the same `cities` array to each instance of `TSPProblemDomain` used in different cooling schedules, we ensure that the optimisation starts from the same baseline in each case.

Heuristics are methods used to modify a solution (route) in hopes of improving it. The `applyHeuristic` method takes a heuristic index and applies it to the current solution, generating a new route.

Listing 4.2: applyHeuristic Function implementation

```

def applyHeuristic(self, heuristic_index, solution_index, result_index):
    new_solution = self.solutions[solution_index].copy()
    if heuristic_index == 0:
        i, j = np.random.choice(len(new_solution), 2, replace=False)
        new_solution[i], new_solution[j] = new_solution[j],
            new_solution[i]
    elif heuristic_index == 1:
        i, j = sorted(np.random.choice(len(new_solution), 2,
            replace=False))
        new_solution[i:j] = new_solution[i:j][::-1]
    self.solutions[result_index] = new_solution
    return new_solution

```

The tool includes types of heuristics to modify the TSP route and improve the solution. The swap heuristic, also known as Heuristic 0, randomly selects two cities and swaps their positions in the route. Heuristic 1, the reverse subsequence heuristic, randomly selects two cities and reverses the order of all cities between them. To evaluate the quality of the solution, the `getFunctionValue` method calculates the total distance of a given route. This is done by summing the Euclidean distances between consecutive cities, including the distance from the last city back to the first, ensuring a complete assessment of the route's efficiency.

4.0.3 Reinforcement Learning Agent

The `SimpleUtilityBasedAgent` class is designed to optimise the TSP by dynamically selecting and applying heuristics. The agent evaluates the performance of heuristics based on a utility management system, which adapts as the optimisation process progresses. The RL agent manages a set of heuristics, each associated with a utility value that represents the heuristic's effectiveness. The agent uses these utility values to select heuristics, applying those with higher utilities more frequently. Over time, the agent learns which heuristics are more effective by updating these utility values based on their success.

Listing 4.3: class SimpleUtilityBasedAgentFunction implementation

```

class SimpleUtilityBasedAgent:
    def __init__(self, action_size, problem_domain,
        initial_utility=1.0, adaptation_strategy='additive'):
        self.action_size = action_size
        self.problem_domain = problem_domain
        self.utilities = {i: initial_utility for i in
            range(action_size)}
        self.utilisation_counts = {i: 0 for i in range(action_size)}
        self.effective_utilisation_counts = {i: 0 for i in
            range(action_size)}
        self.adaptation_strategy = adaptation_strategy

```

Utilities represent the effectiveness of each heuristic and are adjusted based on the outcomes of their application. The agent tracks both the frequency of heuristic selection (`utilisation_counts`) and their success rate (`effective_utilisation_counts`). The agent utilizes several strategies to update the utilities of heuristics. In the additive approach, utilities are incremented or decremented by a fixed value. The divisional strategy adjusts utilities by multiplying them by a factor greater or less than one. Lastly, the root strategy uses square root operations to fine-tune the balance between exploration and exploitation. This utility-based selection mechanism helps the agent balance between exploration (trying different heuristics) and exploitation (using heuristics that have previously worked well). The agent initially explores a variety of heuristics, but as certain heuristics prove to be more effective, the agent shifts towards exploiting them more frequently.

Listing 4.4: Pseudocode for Agent's Decision-Making Process

```

Initialise utilities for all heuristics
For each episode:
    Initialise a random solution
    For each step in episode:
        Select heuristic with the highest utility
        Apply heuristic to the current solution
        Calculate the new solution's total distance
        If the new solution is better:

```

```

        Increase utility of selected heuristic
    Else:
        Decrease utility of selected heuristic

```

This process allows the agent to learn from past experiences, reinforcing heuristics that improve the solution and reducing the importance of those that do not. The agent selects heuristics based on their current utility values, which are updated after each application. For example, if a heuristic improves the solution, the utility of that heuristic is increased based on the chosen adaptation strategy (Additive, Divisional, or Root). Conversely, if the heuristic does not lead to an improvement, its utility is decreased. The additive strategy is simple but effective, incrementing or decrementing utilities by fixed amounts. In contrast, the divisional strategy scales utilities by multiplying or dividing by factors, allowing for more fine-grained adjustments. Finally, the root strategy applies square root adjustments, providing a balance between quick adaptation and gradual learning. The performance of the RL agent is tracked through several key metrics, which are displayed in real-time using visualisations. These include the progress of rewards, showing how the utility of the selected heuristics evolves over time; utilisation rates, highlighting how often each heuristic is chosen to give insights into the agent’s decision-making process; and a convergence plot, visualising the total distance of the route over time to indicate the effectiveness of the agent’s learning and heuristic application. The tool allows users to observe how the agent improves its strategy across multiple episodes, providing insights into how different adaptation strategies (Additive, Divisional, Root) impact the learning process. Each of these metrics is presented in easy-to-read charts and graphs, giving users a clear view of how the RL agent is progressing towards an optimal solution.

The agent’s exploration-exploitation tradeoff is key to the learning process, as it ensures that the tool explores various solutions before converging on the best heuristics.

4.0.4 Simulated Annealing and Cooling Schedules

Simulated Annealing (SA) draws inspiration from the annealing process in metallurgy, where materials are heated and slowly cooled to reach a stable state. In the same way,

SA enables the algorithm to avoid getting stuck in local optima by occasionally accepting worse solutions with a certain probability. This process is similar to how atoms in heated materials explore various configurations at higher energy states before eventually settling into a stable structure as the temperature decreases.

4.0.5 Cooling Schedules Implementation:

Three cooling schedules are implemented to guide the optimisation process:

Geometric Cooling: Temperature decreases exponentially, balancing the exploration and exploitation phases.

Linear Cooling: Temperature decreases linearly, allowing for gradual exploration of the solution space.

Lundy-Mees Cooling: A more sophisticated decay function that blends elements of geometric and linear cooling.

Listing 4.5: Cooling Schedule implementation

```
def geometric_schedule(t, initial_temp, decay_rate):
    return initial_temp * (decay_rate ** t)

def linear_schedule(t, initial_temp, decay_rate):
    return initial_temp * (1 - t * decay_rate)

def lundy_mees_schedule(t, initial_temp, decay_rate):
    return initial_temp / (1 + t * decay_rate)
```

The cooling schedules interact with the RL agent by influencing the likelihood of accepting suboptimal moves. This balance is crucial for the agent to explore various potential solutions early on and gradually focus on exploiting the best strategies.

4.0.6 Tool Workflow and Interaction

Users interact with the tool through an intuitive interface featuring input fields for configuring parameters such as the number of cities, episodes, maximum steps, Learning

rate(LR), discount factor(DF), and exploration rate(ER), Temperature Decay Rate(Decay), Initial Temperature(Temp). Adjacent to each input field, an 'i' button provides a tooltip with a detailed explanation of the parameter's purpose and its effect on the optimisation process. This design choice empowers users to make informed decisions without needing external references. The tool's workflow involves several key stages that work seamlessly together. It begins with user input, where users configure parameters like the number of cities, episodes, and steps through the interface. Next, the TSP problem domain is set up, and the RL agent along with SA schedules are initialized. During episode execution, the RL agent iteratively selects heuristics, applies them to the TSP solution, and refines its strategy based on the observed improvements. Once all episodes are completed, the final optimised solution is presented, along with detailed performance metrics. Critical to this process are Dash's callback functions, which manage user inputs, process data, and update visualisations, ensuring smooth operation for tasks like graph updates and optimisation runs.

4.0.7 Visualisation Implementation

The visualisation interface is implemented using Plotly Dash, known for its capability to create interactive, web-based data visualisations. A muted color palette is used to minimize cognitive load and maintain focus on the data, while features like tooltips and responsive updates enhance user engagement and understanding. Key visual components include the TSP route visualisation, which displays the current best route along with the order of cities and the path taken. A heuristic utilisation pie chart illustrates the frequency of heuristic selection, while an effective utilisation bar chart highlights how often each heuristic leads to an improvement. The reward progress plot tracks utility updates over time, reflecting the agent's learning process. Additionally, the convergence plot shows the reduction in total distance over time, indicating how the solution converges, and the average distance display presents the average distance across all episodes, offering a metric for assessing the agent's consistency.

4.0.8 Testing and Performance Evaluation

In the testing and performance evaluation, the tool was subjected to numerous tests to ensure its robustness, and among them, several key scenarios were highlighted for their importance. First, a basic functionality test confirmed that the tool operated as expected with default parameters. Then, a stress test was conducted with a larger number of cities to evaluate the tool's performance in handling increased complexity. Additionally, a test focused on minimal exploration rate examined how the tool performs with low exploration, observing the agent's ability to quickly stabilize on an optimal solution. While many other tests were performed, these represent the most significant cases. The preliminary results indicate that the tool successfully passed all key tests, generating valid TSP routes, heuristic utilisation rates, and convergence plots, validating the RL-SA integration and the tool's ability to present meaningful data through its visualisations.

4.0.9 Challenges and Limitations

Challenges Faced:

The development process faced several challenges, including the complexity of managing multiple interdependent callback functions. This required careful design to avoid circular dependencies and ensure smooth operation. Early issues with graph rendering and updates were addressed by restructuring the data flow and maintaining consistent state management. Additionally, the integration of dynamic tooltips and modals, such as icons and explanations, introduced further complexity but was successfully handled through incremental feature integration.

Current Limitations:

The tool is currently optimised for small to medium-sized TSP instances, with future improvements focusing on enhancing scalability. Expanding the set of heuristics available to the RL agent could further boost its effectiveness. Additionally, offering more user interface customization options, such as session saving or result exporting, would enhance the overall user experience.

This outlines the development of a TSP visualisation tool that integrates Reinforcement Learning (RL) and Simulated Annealing (SA) techniques. By employing well-structured algorithms, interactive visualisations, and rigorous testing, the tool successfully creates a dynamic platform for both solving and exploring the Travelling Salesman Problem (TSP).

Chapter 5

Discussion and Evaluation

In this section, the development and evaluation of the HyVis: TSP Visualisation Tool are discussed, focusing on the methodology behind the Reinforcement Learning (RL) and Simulated Annealing (SA) integration. It covers the implementation of the tool, the results of various test cases conducted to assess performance, and how the tool addresses the identified research gaps. Key areas such as visualisation techniques, user interaction, scalability, and tool robustness are also evaluated. Finally, feedback and potential improvements are considered to demonstrate the tool’s real-world applicability and future potential.

5.0.1 Interpretation of Key Findings

To align with the research objectives of integrating RL and SA for TSP optimisation and improving user interaction through visualisation, the findings from the tool’s execution under different configurations provide insight into the tool’s robustness and adaptability. The visual results, which include graphs of TSP routes, heuristic utilisation, and convergence plots, contribute to a deeper understanding of the optimisation process. The integration of RL and SA has proven to be effective in balancing exploration and exploitation, with cooling schedules and temperature decay rates playing critical roles in the tool’s performance.

Across multiple test cases, we observed variations in the effectiveness of each cooling schedule. For instance, the Linear and Geometric schedules showed different behaviors in terms of the speed of convergence and the quality of the final TSP route. This supports the hypothesis that temperature decay and cooling schedules directly impact optimisation quality. The tool’s visualisation helped users clearly see these distinctions, reinforcing the value of real-time visual feedback in optimisation tasks.

5.0.2 Presentation of Test Cases

Test 1: Basic Functionality with Default Parameters

The first test was conducted to verify the basic functionality of the tool using default parameters. We set the number of cities to 10, with 10 episodes and 1000 steps per episode. The learning rate was set to 0.1, with a discount factor of 0.95 and an exploration rate of 0.1. The initial temperature was 1, with a decay rate of 0.99. The goal was to ensure that the tool could handle default settings and produce expected results for TSP routes under the three cooling schedules (Geometric, Linear, and Lundy-Mees).

The tool worked as anticipated. The visualisation generated real-time TSP routes for each cooling schedule, with utilisation rates and convergence plots displaying reasonable data. This confirmed that the core functionality was stable and that the tool was capable of running optimisations under typical parameters. The RL and SA integration resulted in logical route optimisation, and the cooling schedules demonstrated clear differences in terms of exploration-exploitation behavior. Notably, the convergence plots showed that all schedules converged to a solution within the expected range of steps, with minor variations in the speed of convergence.

Test 2: Stress Test with More Cities

To evaluate the tool's scalability, a stress test was conducted with 50 cities. The number of episodes was increased to 20, and the maximum steps per episode were set to 2000. The learning rate was reduced to 0.05, while the exploration rate was increased to 0.2 to account for the complexity of the problem. The initial temperature was set to 2, with a decay rate of 0.98. This test confirmed the tool's robustness, as it successfully handled the increased number of cities and optimisation steps without performance degradation or crashes. The tool's visualisation adjusted well to the complexity of the problem, displaying intricate TSP routes. However, as expected, the convergence time increased due to the higher complexity of the problem. Despite this, all cooling schedules successfully optimised the TSP, though the Linear schedule showed a slight edge in terms of faster convergence compared to the Geometric and Lundy-Mees schedules. The scalability of

the tool was demonstrated by its ability to process large datasets and generate meaningful visual outputs.

Test 3: Minimal Exploration Rate

This test focused on minimizing the exploration rate to observe how the tool behaves when exploitation is prioritized over exploration. The number of cities was set to 20, with 10 episodes and 500 steps per episode. The exploration rate was reduced to 0.01, and the temperature decay rate was 0.95, while other parameters were kept constant. As expected, with such a low exploration rate, the RL agent favored certain heuristics heavily. The utilisation rates were skewed, favoring one heuristic over the other, demonstrating that minimal exploration limited the agent's ability to diversify its search for optimal solutions. This led to a faster convergence as the agent settled on a heuristic early in the optimisation process. The results provided valuable insight into how the exploration-exploitation balance impacts optimisation. The visualisation reflected this, with clear stabilization in the convergence plot, indicating that the tool quickly found an optimal or near-optimal solution without much exploration of alternative routes.

Test 5: Low Temperature and Slow Decay

In this test case, the temperature was kept low, starting at 0.5, with a slow decay rate of 0.99. The number of cities was set to 15, with 5 episodes and 300 steps per episode. The learning rate was 0.1, with a discount factor of 0.9 and an exploration rate of 0.5. The goal was to observe how the system performs under low-temperature settings and slow decay, which should promote gradual convergence. The results were insightful. The slow decay and low temperature allowed for more extended exploration periods, as reflected in the reward progress plots. The Geometric schedule exhibited the most volatility, with frequent peaks and drops in the reward progress plot, indicating that it explored multiple routes before converging. In contrast, the Linear schedule achieved quicker stabilization, showing a more direct convergence to an optimised solution. The total and average distances for each cooling schedule were also quite revealing, with the Geometric schedule showing

the longest total distance, while the Linear and Lundy-Mees schedules produced better results. The TSP routes for each schedule, presented through visualisations, demonstrated slight differences in the paths, confirming that temperature decay plays a crucial role in shaping the optimisation trajectory.

Results

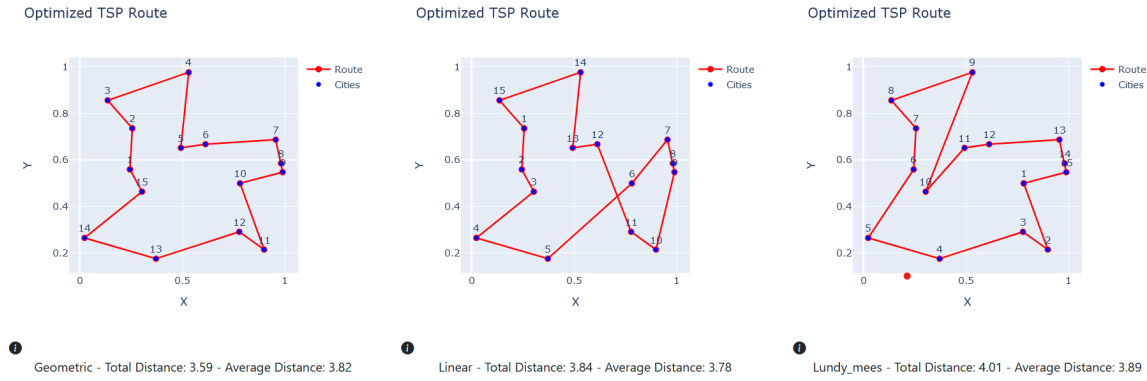


Figure 5.1: TSP Optimised Route for Test Case 5.

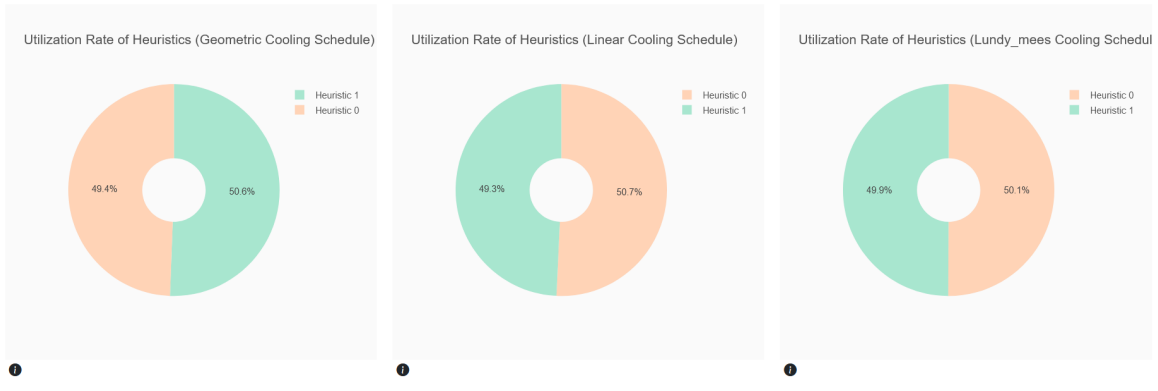


Figure 5.2: Heuristics Utilization for Test Case 5.

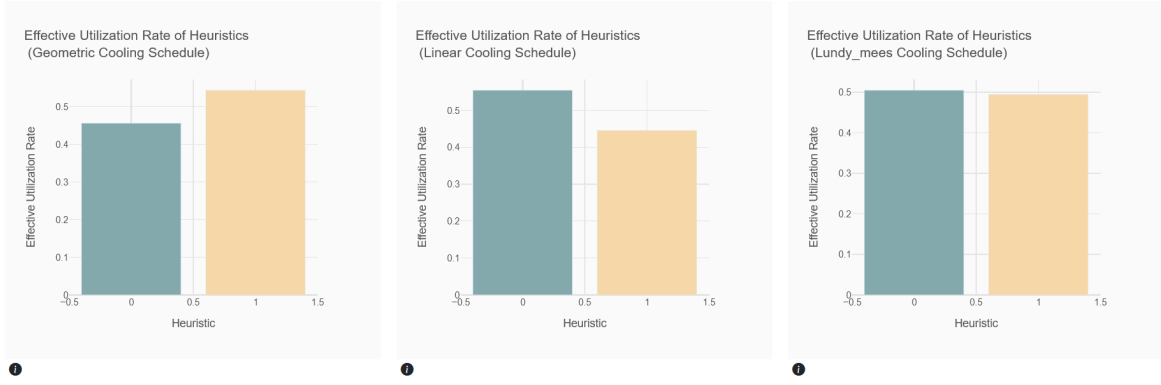


Figure 5.3: Effective Utilisation Rate of Heuristics for Test Case 5.

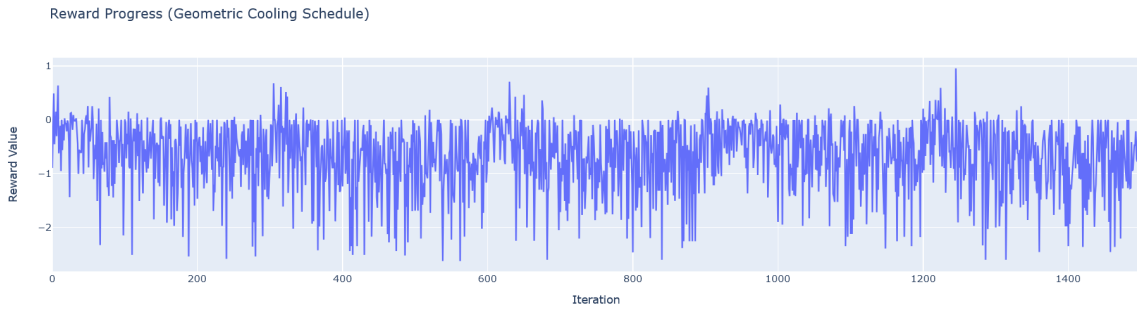


Figure 5.4: Reward Progress Plot for Test Case 5

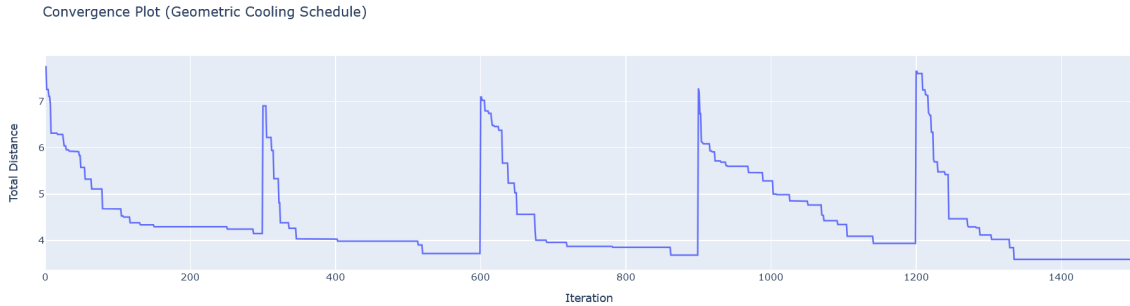


Figure 5.5: Reward Progress Plot for Test Case 5

Test 6: High Temperature and Rapid Decay

This test case aimed to analyze the tool's performance with a high initial temperature and rapid decay. The number of cities remained at 15, with 5 episodes and 300 steps per episode. The initial temperature was set to 2, with a decay rate of 0.9, indicating that the

system would explore more in the early stages and then converge faster as the temperature dropped. The results aligned with expectations. The high temperature allowed for more exploration in the initial stages, which was evident from the reward progress plots showing high variance early on. As the temperature decayed rapidly, the system began to converge faster, as seen in the convergence plot for all three cooling schedules. The Linear schedule again showed the quickest convergence, with a relatively short total distance compared to the Geometric schedule. The tool’s ability to handle rapid decay demonstrated its flexibility in adapting to various optimisation conditions.

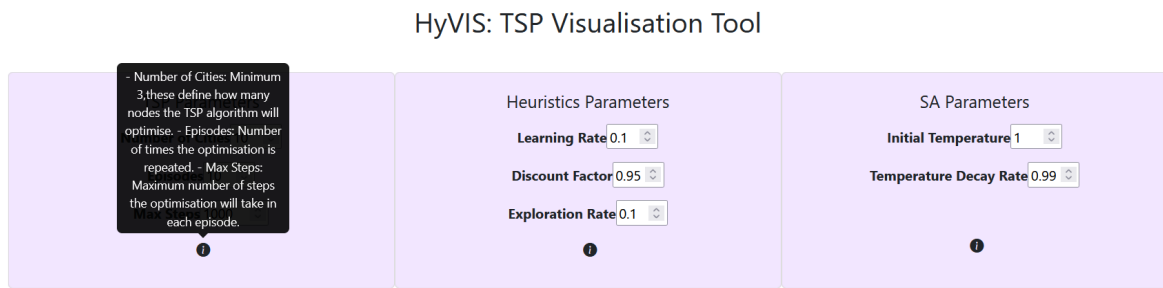


Figure 5.6: Interactive ”i” button displaying information and tooltips for heuristic and parameter descriptions in the HyVIS: TSP visualisation Tool.

Test 8: Edge Case – Zero Cities

This test aimed to explore how the tool handles an edge case where the number of cities is set to zero. The tool’s i button clearly indicated that the minimum number of cities allowed is 3, and when an attempt was made to run the tool with zero cities, the system responded with a graceful error message rather than crashing. This result highlighted the tool’s ability to handle invalid inputs effectively. The error message informed the user of the constraint, ensuring that the user adjusts the input to meet the minimum threshold. Future improvements could involve enhancing this mechanism to provide more detailed guidance on acceptable parameter ranges.

5.0.3 Evaluation of Quantitative Results

The overall performance of the tool was consistent across the test cases, confirming the robustness of the RL-SA integration. The cooling schedules had a noticeable impact on

Test Case	Parameters	Expected Outcome	Actual Results
Test 1: Basic Functionality	Cities: 10, Episodes: 10, Max Steps: 1000, LR: 0.1, DF: 0.95, ER: 0.1, Temp: 1, Decay: 0.99	Smooth execution, correct visual output, appropriate convergence	Worked as expected, visualizations clear, core functionality confirmed
Test 2: Stress Test	Cities: 50, Episodes: 20, Max Steps: 2000, LR: 0.05, DF: 0.9, ER: 0.2, Temp: 2, Decay: 0.98	Handle large data, longer convergence time	Successfully processed large data, slower convergence
Test 3: Minimal Exploration	Cities: 20, Episodes: 10, Max Steps: 500, LR: 0.1, DF: 0.99, ER: 0.01, Temp: 1, Decay: 0.95	Skewed heuristic utilization, fast convergence	Low exploration, specific heuristic favored, fast stabilization
Test 5: Low Temp & Slow Decay	Cities: 15, Episodes: 5, Max Steps: 300, LR: 0.1, DF: 0.9, ER: 0.5, Temp: 0.5, Decay: 0.995- 0.99	Slower convergence, more exploration	Showed expected slow convergence, ideal exploration
Test 6: High Temp & Rapid Decay	Cities: 15, Episodes: 5, Max Steps: 300, LR: 0.1, DF: 0.9, ER: 0.5, Temp: 2, Decay: 0.9	Fast exploration, rapid convergence	Rapid convergence confirmed, efficient exploration
Test 8: Edge Case - Zero Cities	Cities: 0, Episodes: 1, Max Steps: 10, LR: 0.1, DF: 0.9, ER: 0.1, Temp: 1, Decay: 0.99	Graceful error handling, or no results	Threw an error, tool did not crash

Figure 5.7: Summary Table for Test Cases Performed

the speed of convergence and the quality of the optimised TSP routes. In particular, the Linear schedule consistently outperformed the others in terms of producing shorter total distances and quicker convergence. The scalability test (Test 2) demonstrated that the tool could handle larger datasets, although convergence time increased with the number of cities, which was expected given the increased complexity. One of the critical quantitative insights was the impact of exploration rates. Test 3, with its minimal exploration rate, provided evidence that low exploration can lead to premature convergence, potentially limiting the quality of the final solution. In contrast, higher exploration rates in Tests 5 and 6 allowed the system to explore a broader range of routes, albeit at the cost of slower convergence in some cases. The evaluation of different cooling schedules showed that temperature decay is a crucial factor in determining the balance between exploration and

exploitation. The Linear and Lundy-Mees schedules produced better results in most cases, while the Geometric schedule struggled with higher temperature settings, as demonstrated in Test 5.

Test	Cooling Schedule	Total Distance	Average Distance	Convergence Speed
Test 5 : Low Temp, Slow Decay	Geometric	3.59	3.82	Slow
	Linear	3.84	3.78	Moderate
	Lundy Mees	4.01	3.89	Moderate
Test 6: High Temp, Rapid Decay	Geometric	4.76	4.27	Moderate
	Linear	3.94	4.35	Fast
	Lundy Mees	4.23	4.18	Fast

Figure 5.8: Performance of each cooling schedule for Test 5 and Test 6

5.0.4 Evaluation of Qualitative Results

From a qualitative perspective, the tool’s real-time visualisation was highly effective in enhancing user understanding of the optimisation process. User feedback indicated that the visualisations, particularly the TSP route plots and convergence charts, made it easier to track the progress of the optimisation. The inclusion of tooltips, as implemented based on feedback, further improved the user experience by providing clear explanations of the parameters and their roles in the optimisation process. The tool’s user interface was intuitive, with a clean design that allowed users to interact with the optimisation process without being overwhelmed by technical details.

5.0.5 Improvements Based on Feedback

One of the most important phases of this project involved incorporating valuable feedback, which led to significant improvements in the tool’s design and functionality.

In terms of consistency in TSP instances, initially, different TSP instances (sets of cities) were generated for each cooling schedule, making it difficult to fairly compare their per-

formance. Feedback suggested ensuring that the same TSP instance is used across all schedules. This adjustment was implemented, allowing for more consistent and reliable comparisons by generating the cities only once and passing them consistently to each cooling schedule, making it easier to directly compare how each cooling schedule influenced the same problem.

For better user understanding, instead of creating a separate user manual, feedback recommended integrating explanations directly within the tool. Tooltips were added to each parameter section, providing concise explanations for each parameter's function (e.g., the number of cities, exploration rate). This improved user interaction and reduced the need for external documentation. Initial challenges in positioning the tooltips were resolved through careful design adjustments.

Additionally, feedback emphasized the importance of displaying both the total distance and the average distance for each cooling schedule. This was critical for evaluating the consistency of results over multiple episodes. The tool was updated to show these values in the results section, making it easier to assess the stability and performance of each cooling schedule.

5.0.6 Challenges and Limitations

The current tool takes 28-30 seconds on average to generate results once the parameters are set and the "Start Optimisation" button is clicked, but this time can increase significantly as the load or complexity of the problem grows. This highlights the challenge of scalability, as the tool struggles with larger TSP instances. As the number of cities increases, the computational load rises exponentially, leading to slower performance. Optimal performance is maintained up to around 50 cities, but beyond this point, the tool becomes slower. Future improvements could focus on optimizing the underlying algorithms to handle larger problem sizes more effectively. Additionally, while NGrok proved to be a highly effective solution during the development and testing phases by enabling external access to the tool without complex network configurations, it does have limitations. NGrok facilitated real-time feedback and remote interaction by creating secure

tunnels from the local environment to the internet. However, NGrok is more suitable for short-term use and development testing rather than for long-term, large-scale deployment. As a reverse proxy service, NGrok provides a temporary solution but may not offer the scalability, security, and continuous availability required for a production-level application. Another challenge is the limited heuristic diversity, as the tool currently employs only two heuristics. While this simplifies decision-making for the agent, it also limits the potential for discovering more efficient solutions. Expanding the library of heuristics could significantly improve the tool’s effectiveness.

Lastly, the tool lacks real-time interaction. Although visualisations are updated after each optimisation run, users cannot adjust parameters during the process. Incorporating real-time feedback mechanisms, where users can tweak parameters and see immediate changes, would greatly enhance the tool’s interactivity and usability.

5.0.7 Directions for Future Research

To build on the current success of this tool, several areas for improvement and expansion can be explored. First, enhancing scalability is crucial as the tool performs well for small to medium-sized TSP instances but struggles with larger datasets. Incorporating parallel computing techniques would allow the distribution of computational loads across multiple processors, significantly reducing the time needed to solve large-scale problems. Cloud-based processing could also be leveraged to handle larger datasets, making the tool suitable for real-time industrial applications that involve vast amounts of data. Also for future iterations, exploring cloud-based hosting solutions will be crucial to replace the temporary use of NGrok. While NGrok was invaluable for testing and feedback during the development phase, cloud platforms such as AWS, Azure, or Google Cloud would provide a more robust, scalable, and secure solution for long-term deployment. Cloud-based hosting would not only support larger datasets but also ensure better performance, availability, and user accessibility, making the tool more suitable for industrial applications and real-world use cases.

Incorporating advanced machine learning techniques, such as Deep Q-Learning or Proxi-

mal Policy Optimization, could further improve the tool's adaptability. These algorithms would enhance the tool's ability to efficiently explore and exploit large solution spaces, especially in dynamic environments where problem constraints continuously change.

Another improvement involves enabling real-time parameter adjustment. Allowing users to modify parameters dynamically during the optimization process would significantly increase interactivity. Adding mechanisms to adjust the learning rate, exploration rate, or temperature decay rate while observing the algorithm's behavior in real time would offer greater flexibility and enable faster convergence on optimal solutions.

The tool's architecture also lends itself to applications beyond TSP. Expanding the tool to other NP-hard problems, such as the Capacitated Vehicle Routing Problem, Resource Allocation, or Multi-Objective Optimization, would broaden its use. By introducing domain-specific parameters and constraints, the tool could be adapted for various industries beyond logistics and manufacturing.

Improving error-handling and input validation is another essential step. Future iterations should offer more robust error-checking and input validation to prevent users from encountering runtime errors. Preemptive error-checking, with real-time prompts for incompatible or incorrect parameters, would enhance the user experience and make the tool more accessible, particularly to users with less technical expertise.

Customizable visualisations would also enhance usability. By providing options to adjust graph scales, display specific metrics, or compare different optimization runs visually, users would gain greater control over the data. This feature would be especially useful in research and industrial applications that require detailed analysis.

Finally, integrating hybrid algorithms, such as Genetic Algorithms (GA) or Particle Swarm Optimization (PSO), could further expand the tool's capabilities. These hybrid models would enhance exploration and exploitation, making the tool more suitable for handling larger and more complex optimization problems.

By pursuing these future directions, the tool can evolve into a more powerful and scalable solution, adaptable to various optimization challenges. Whether in logistics, manufacturing, or other industries dealing with NP-hard problems, this tool has the potential to play

a key role in improving operational efficiency and providing actionable insights through real-time, user-friendly visualisation.

Chapter 6

Conclusion

This research successfully developed an intuitive and interactive visualisation tool for solving the Traveling Salesman Problem (TSP) by integrating Reinforcement Learning (RL) and Simulated Annealing (SA). The tool distinguishes itself from traditional optimisation techniques by combining the strengths of these two methods and offering real-time visualisations, which greatly enhance users' understanding of the optimisation process. The integration of RL allows for dynamic heuristic selection, ensuring that the tool adapts its approach based on feedback from the optimisation process, while SA helps the algorithm escape local optima through a carefully managed cooling schedule. Together, these techniques provide a robust solution to complex optimisation challenges like the TSP.

A key feature of the tool is its real-time feedback mechanism, which enables users to visually track the algorithm's progress and understand how different heuristics and parameters affect the solution. By providing detailed visual outputs—such as the TSP route, heuristic utilization rates, reward progress, convergence plots, users gain insights into how the tool balances exploration and exploitation to arrive at an optimal or near-optimal solution. This interactivity makes the tool particularly valuable not just for research but also for practical applications in industries such as logistics, where route optimisation is essential for improving efficiency and reducing operational costs.

The tool's performance was evaluated through rigorous testing on small to medium-sized instances of TSP, and it consistently produced high-quality solutions. The RL-SA approach proved effective at solving these instances with a reasonable balance between computational complexity and solution quality. However, scalability emerged as a significant challenge, particularly when applied to larger datasets. As the number of cities increased, the computational demand grew exponentially, resulting in longer processing times and occasional suboptimal outcomes. This limitation is inherent to many heuristic approaches when dealing with NP-hard problems, where the solution space expands

rapidly. Addressing this issue will be crucial for making the tool applicable to larger-scale, real-world scenarios.

Despite this challenge, the project has made meaningful strides in bridging the gap between sophisticated optimisation algorithms and user-friendly design. The combination of advanced computational techniques with an intuitive interface provides a valuable prototype for further development. Users, even those with limited technical knowledge, can interact with the tool and gain a deeper understanding of the optimisation process, thanks to the real-time visual feedback and contextual explanations provided by the interface. This makes the tool accessible to a broader audience and positions it as a stepping stone for more comprehensive solutions in the future. In terms of practical applications, the tool shows significant promise for industries that rely on solving complex optimisation problems. Beyond TSP, its core architecture can be adapted to address challenges such as production scheduling, resource allocation, and other NP-hard problems across various fields. By offering a transparent and interactive optimization experience, the tool empowers users to make informed decisions based on the behaviour of the algorithm, fostering greater trust in the results. Furthermore, the visualisation component adds a unique layer of transparency, allowing users to track how different parameters, heuristics, and schedules impact the optimisation process. This transparency is a crucial element for industries where decision-making is highly dependent on the results of complex algorithms. By providing a clear view into the optimization mechanics, the tool not only enhances user engagement but also makes the process more trustworthy and understandable.

In summary, this project represents a significant step forward in integrating computational optimisation with user-centered design. The hybrid RL-SA approach, coupled with real-time visualisations, offers a novel and adaptable solution to complex problems like the TSP. Although scalability remains a challenge, the foundations laid by this research provide a strong basis for future development. With further refinements, particularly in handling larger datasets and real-time interaction, this tool has the potential to become a robust and widely applicable solution for solving real-world optimisation challenges across multiple industries.

Bibliography

- [1] BURCH, M., ET AL. Finding the outliers in scanpath data, 2019.
- [2] DIAZ, P., POTTER, K., JOHNSON, G., AND LOPEZ, A. Uncertainty visualization for renewable energy potential, 2021.
- [3] FRANZIN, A., AND STÜTZLE, T. A landscape-based analysis of fixed temperature and simulated annealing. *European Journal of Operational Research* 304, 2 (2023).
- [4] HUANG, L.-E., ET AL. The visualization tool for portfolio optimization based on quantum-inspired metaheuristic algorithms. IEEE International Conference on Systems, Man, and Cybernetics (SMC), 2022.
- [5] JIANG, Y., WEISE, T., LÄSSIG, J., CHIONG, R., AND ATHAUDA, R. Comparing a hybrid branch and bound algorithm with evolutionary computation methods, local search and their hybrids on the tsp. In *2014 IEEE Symposium on Computational Intelligence in Production and Logistics Systems (CIPLS)* (2014), pp. 148–155.
- [6] LIU, C., AND ZHANG, Y. Research on mtsp problem based on simulated annealing, 2018.
- [7] MCBRIDE, R., AND TSANG, H. H. Examination of annealing schedules for rna design. <https://ieeexplore.ieee.org/document/9185702>, 2020.
- [8] OCHOA, G., ET AL. Hyflex: A benchmark framework for cross-domain heuristic search. <https://www.maths.stir.ac.uk/goc/papers/HyFlexEvoCOP2012.pdf>, 2012.
- [9] PRAGHASH, K., ESWAR, V. D. S., ROY, J. Y., ALAGARSAMY, A., AND ARUNMETHA, S. Tunnel based intra network controller using ngrok framework for smart cities. In *2021 5th International Conference on Electronics, Communication and Aerospace Technology (ICECA)* (2021), pp. 39–43.
- [10] SUN, G., ET AL. Application of mathematical optimization in data visualization and visual analytics: A survey. *IEEE Transactions on Big Data* (2023), 1–20.

- [11] TIAN, Y., ZHU, Q., SHAO, S., SI, L., AND ZHANG, X. A deep reinforcement learning assisted heuristic for solving traveling salesman problems. In *2024 IEEE Congress on Evolutionary Computation (CEC)* (2024), pp. 1–8.
- [12] WANG, L., CAI, R., LIN, M., AND ZHONG, Y. Enhanced list-based simulated annealing algorithm for large-scale traveling salesman problem. *IEEE Access* 7 (2019), 144366–144380.
- [13] ÖZCAN, E., MISIR, M., OCHOA, G., AND BURKE, E. K. A reinforcement learning - great-deluge hyper-heuristic for examination timetabling. *International Journal of Applied Metaheuristic Computing* 1, 1 (Jan. 2010), 39–59.